



UNIVERSIDAD DE ALMERÍA

Grado en Ingeniería Informática

Introducción a la Programación

2025-2026



Tema 1. Fundamentos de Programación

- Algoritmos y programas
- Introducción a la programación
- Datos y tipos de datos
- Tipos de datos primitivos en Java
- Variables
- Expresiones y sentencias
- Programas
- Estructuras de control
- Uso de subprogramas
- Métodos

Tema 1. Fundamentos de Programación



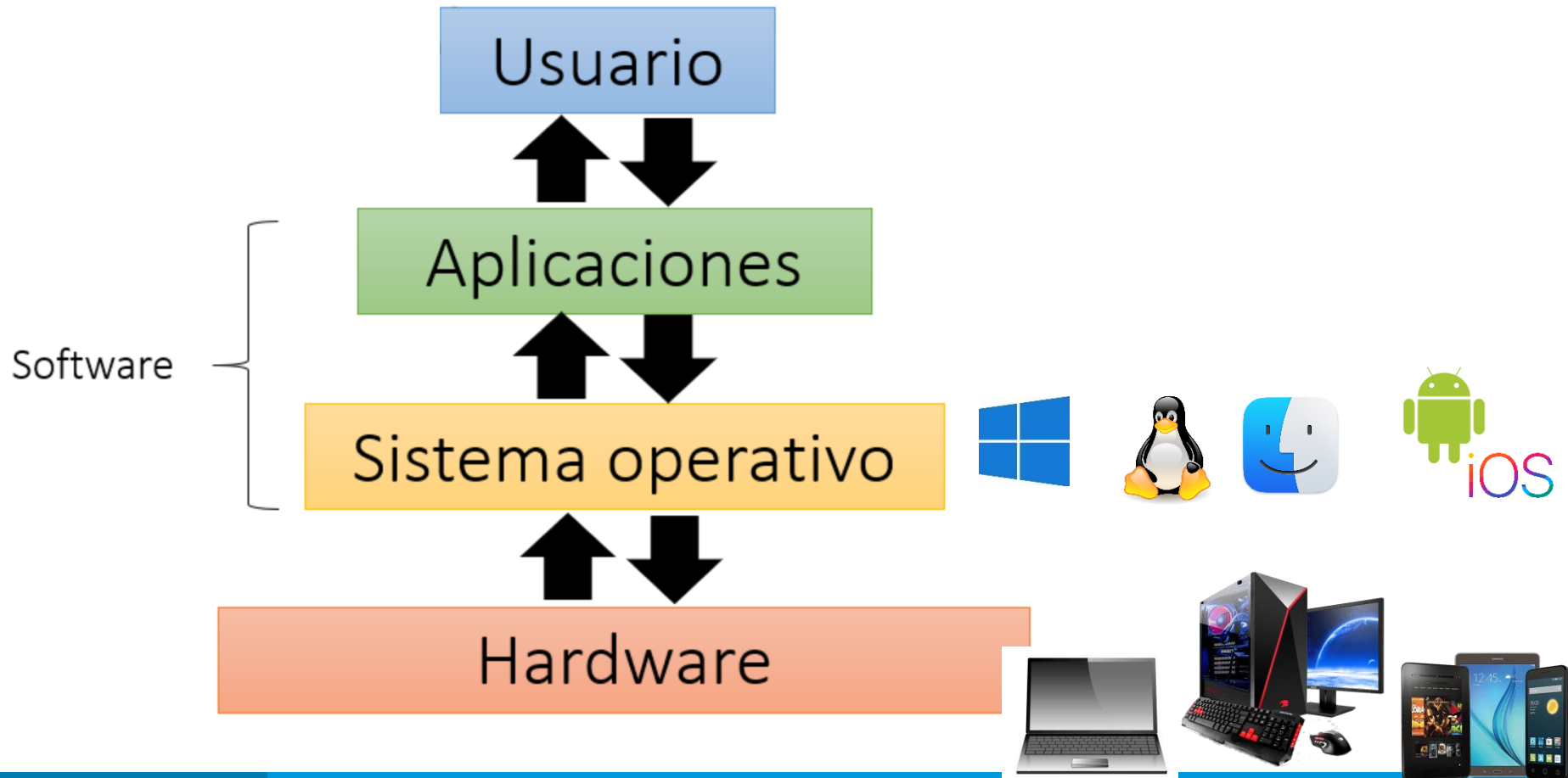
Algoritmos y programas



- ☐ Introducción
- ☐ Algoritmo
- ☐ Programa
- ☐ Lenguajes de programación
- ☐ Traductores
- ☐ Herramientas de programación
- ☐ Ciclo de vida del software

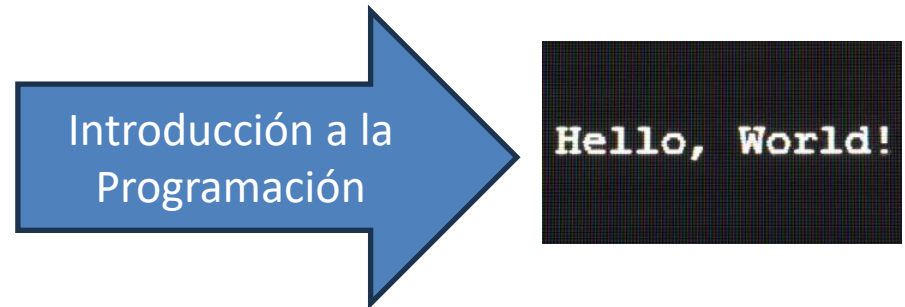
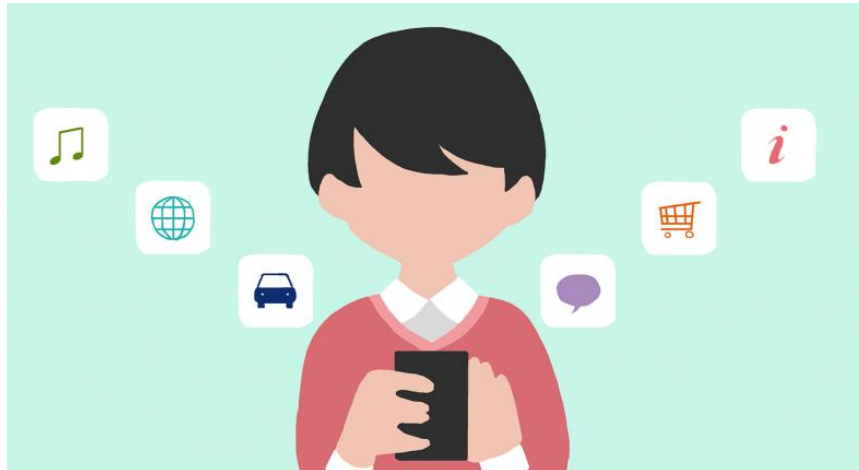
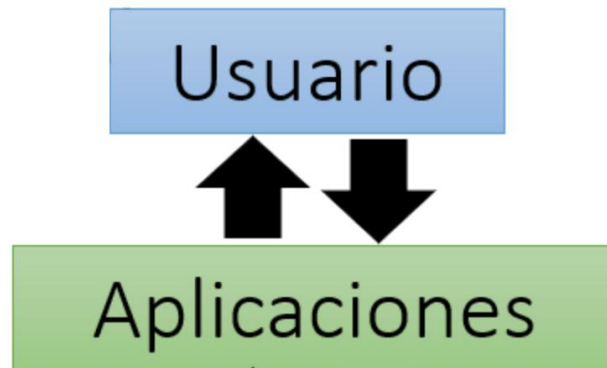
Tema 1. Fundamentos de Programación

➤ Introducción: Software, programas y aplicaciones



Tema 1. Fundamentos de Programación

➤ Introducción: Usuario y aplicaciones



Tema 1. Fundamentos de Programación

Algoritmo

Secuencia ordenada de pasos, bien definidos, que resuelve un problema concreto.

Ejemplos

➤ Algoritmo



40 a 60 segundos

0



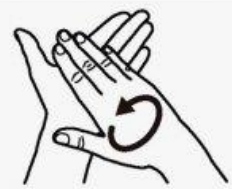
Humedecer las manos con agua

1



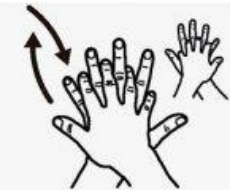
Aplicar suficiente jabón para cubrir la superficie de ambas manos

2



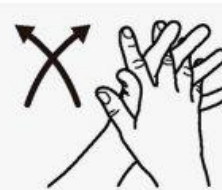
Frotar las palmas de las manos entre sí

3



Frotar la palma derecha sobre el dorso de la izquierda, entrelazando los dedos y viceversa

4



Frotar palma contra palma, entrelazando los dedos

5



Frotar el dorso de los dedos contra la palma de la mano opuesta, agarrándose los dedos

6



Frotar con un movimiento de rotación el pulgar, atrapándolo con la palma de la otra mano, y viceversa

7



Frotar la punta de los dedos de una mano contra la otra palma, haciendo un movimiento de rotación, y viceversa

8



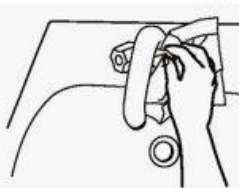
Enjuagarse las manos con agua

9



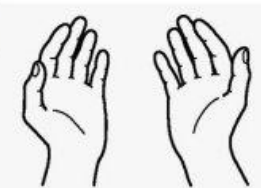
Secarse con una toalla de un solo uso

10



Utilice la toalla para cerrar el grifo

11



Las manos ya son seguras

Ejemplos

Tema 1. Fundamentos de Programación

➤ Algoritmo

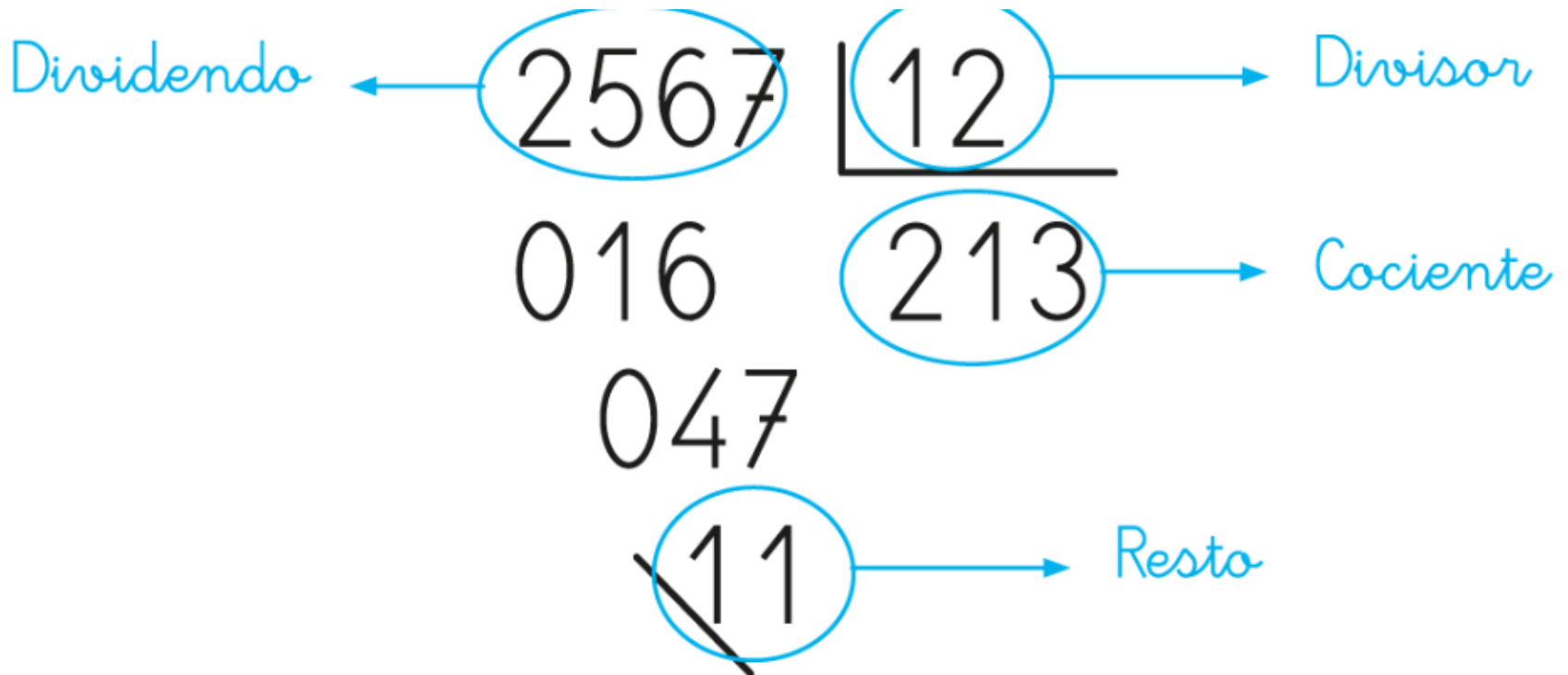


Diagram illustrating the long division algorithm for 2567 divided by 12:

- Dividendo** (Dividend): 2567
- Divisor** (Divisor): 12
- Cociente** (Quotient): 213
- Resto** (Remainder): 11

The division process is shown as follows:

$$\begin{array}{r} 2567 \\ 12 \overline{) 2567} \\ \underline{016} \\ 047 \\ \underline{11} \end{array}$$

Tema 1. Fundamentos de Programación

➤ Algoritmo

Secuencia ordenada de pasos, bien definidos, que resuelve un problema concreto.

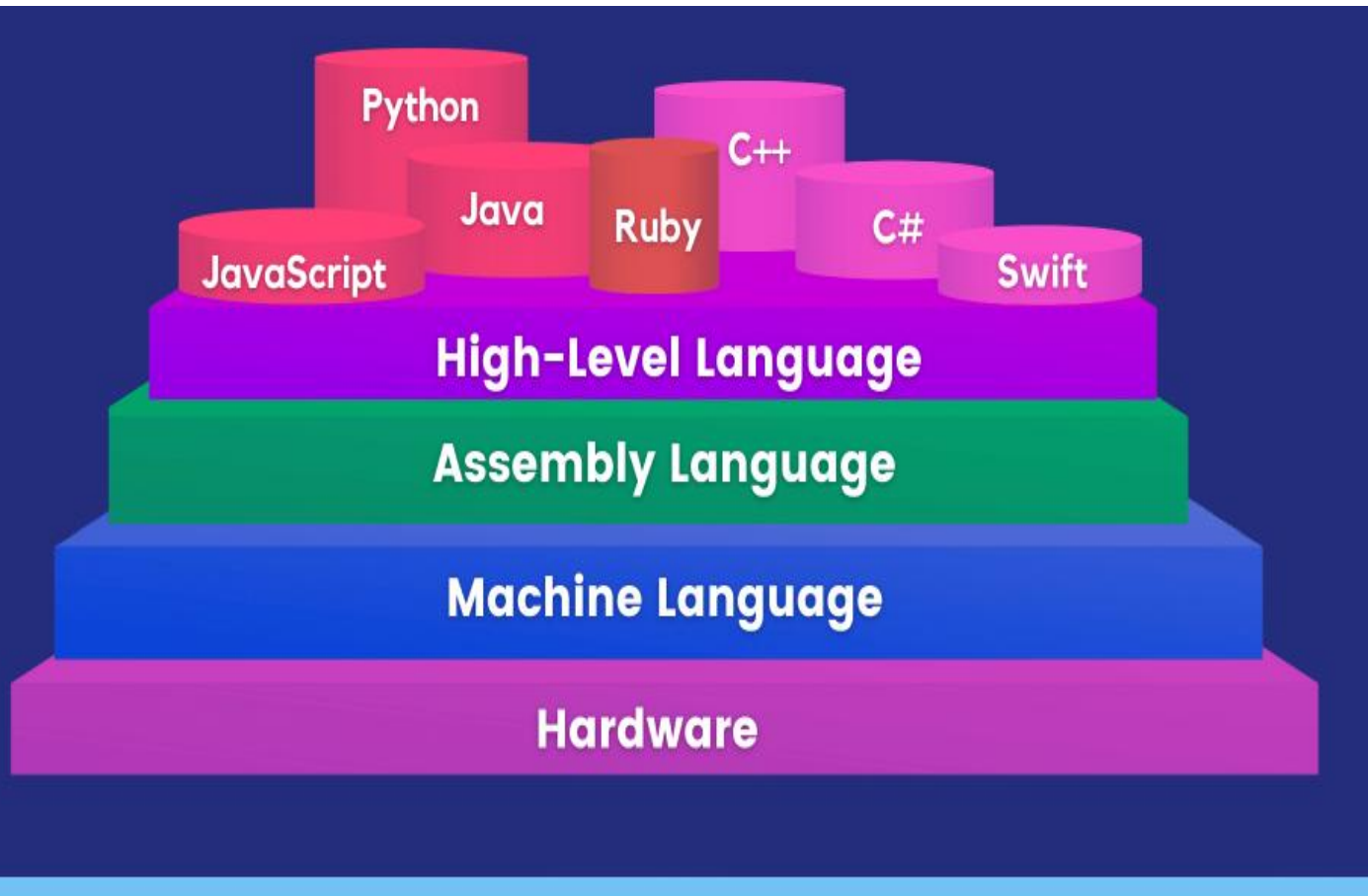
Características

- ✓ Corrección (sin errores).
- ✓ Precisión (ausencia de ambigüedades).
- ✓ Repetitividad (solución genérica de un problema dado).
- ✓ Finitud (número finito de órdenes).
- ✓ Eficiencia (temporal [tiempo necesario] y espacial [memoria utilizada])

Tema 1. Fundamentos de Programación

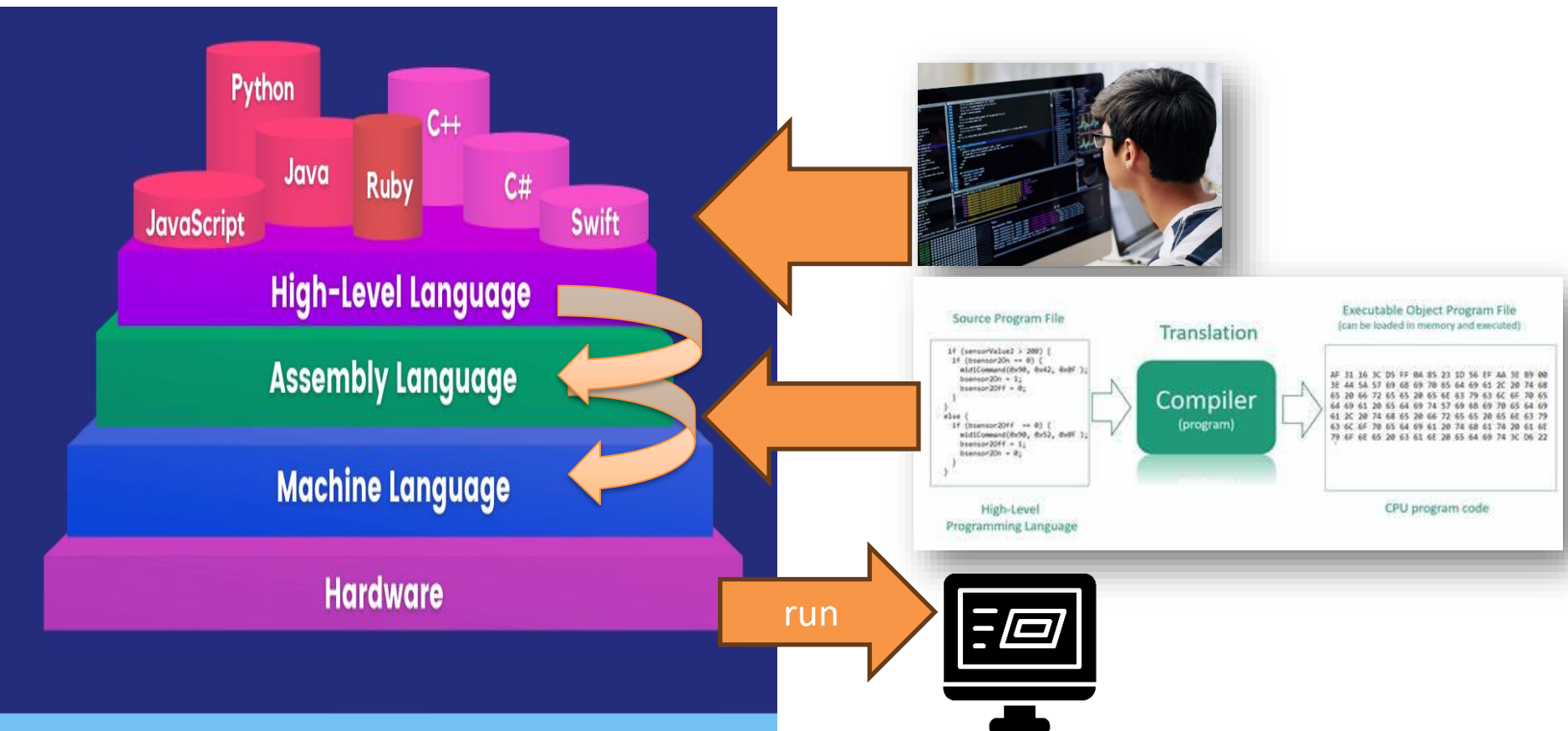


Programa: ¿Cómo escribir un algoritmo?



Tema 1. Fundamentos de Programación

Programas: Niveles de Lenguajes de programación



Tema 1. Fundamentos de Programación

➤ Programa

Implementación de un algoritmo en un lenguaje de programación



Tema 1. Fundamentos de Programación

➤ Programa

Implementación de un algoritmo en un lenguaje de programación

Ejemplo: Programa en Java

Código fuente

```
3 public class Circunferencia {
4
5     public static void main(String[] args) {
6         double radio = 4.5;
7         double longitud = 2 * Math.PI * radio;
8         double area = Math.PI * Math.pow(radio, 2.0);
9
10        System.out.println("Radio = " + radio);
11        System.out.println();
12        System.out.printf("Longitud de la circunferencia = %.4f", longitud);
13        System.out.println();
14        System.out.printf("Area de la circunferencia = %.4f", area);
15        System.out.println();
16    }
17 }
18
```

Tema 1. Fundamentos de Programación

➤ Programa

Conjunto ordenado de instrucciones que se dan al ordenador indicándole las operaciones o tareas que ha de realizar para resolver un problema.

Una **instrucción** es un conjunto de símbolos que representa una **orden para el ordenador**: la ejecución de una operación con datos

Tema 1. Fundamentos de Programación

➤ Variable

- Una **variable** es un **espacio de almacenamiento** asignado en la memoria del sistema que permite **guardar datos** que pueden ser **usados y manipulados** durante la ejecución del programa
- Una variable tiene :
 - un **tipo** de dato
 - un **nombre** (o **identificador**)

```
double radio = 4.5;  
double longitud = 2 * Math.PI * radio;  
double area = Math.PI * Math.pow(radio, 2.0);
```

Tema 1. Fundamentos de Programación

➤ Variable

- **Identificador:** nombre utilizado para identificar variables, métodos, clases, etc.
- Las reglas para escribir correctamente un identificador en Java son:
 - Debe comenzar con una **letra**
 - No puede comenzar con un número.
 - Puede contener **letras** (mayúsculas y minúsculas), **dígitos** (0-9), **subrayados** (_) y signos de dólar (\$)
 - Ejemplos válidos: **nombre**, **_miVariable**, **\$valor**.
 - Ejemplos no válidos: **1variable**, **9num**, **nombre-apellido** (no se permiten guiones), **mi variable** (no se permiten espacios).
 - **Distingue entre mayúsculas y minúsculas.**
 - **No puede ser una palabra reservada en Java:** *class, if, else, public, static, etc.*
 - No hay límite de longitud, recomendable que sean descriptivos y no excesivamente largos.

Tema 1. Fundamentos de Programación

➤ Lenguajes de programación

Idiomas artificiales diseñado para que sea fácilmente entendible por un humano e interpretable por una máquina

Se forman con **símbolos** tomados de un determinado repertorio (componentes léxicos)

Se construyen siguiendo unas **reglas** precisas (**sintaxis**)

- Lenguaje máquina
- Lenguaje ensamblador
- Lenguajes de alto nivel

Tema 1. Fundamentos de Programación

➤ Lenguajes de programación

Clasificación de lenguajes de programación de alto nivel

❖ Lenguajes imperativos:

Los programas indican al ordenador de forma inequívoca los pasos a seguir para la resolución de un problema.

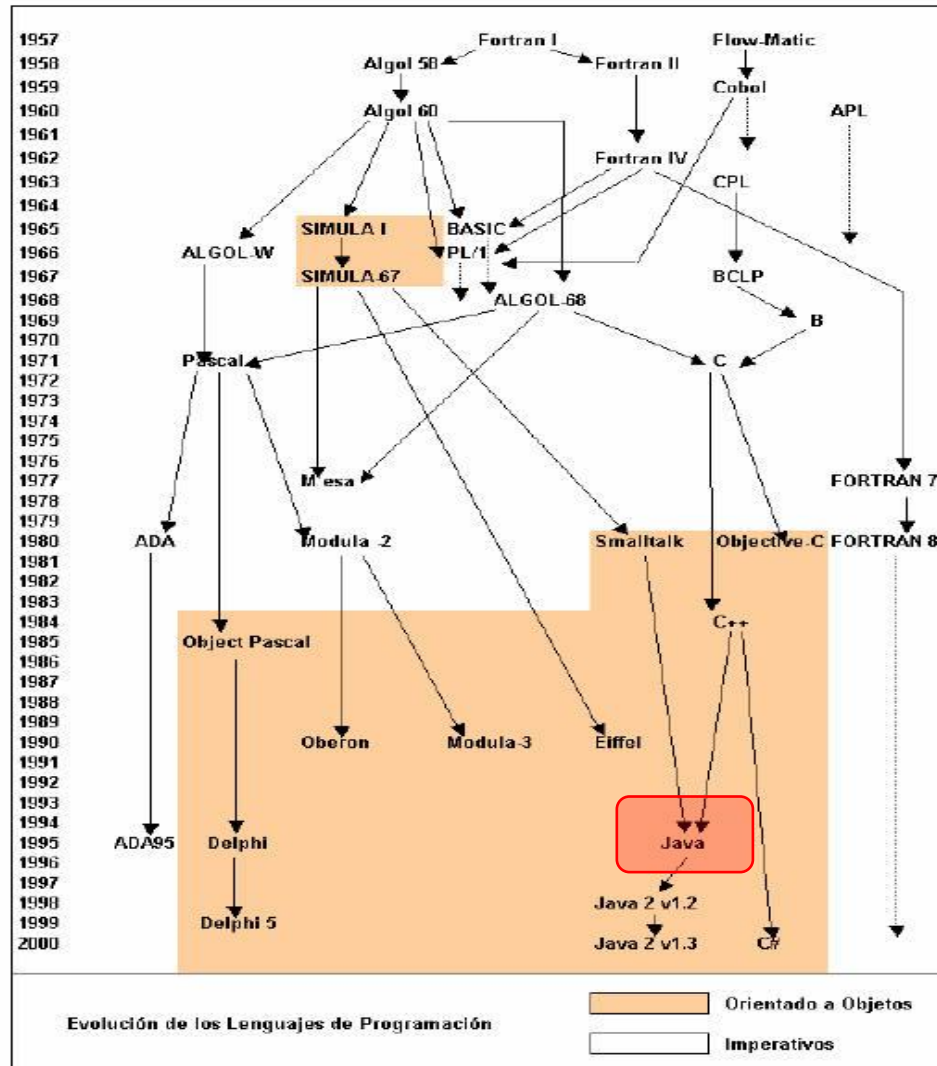
- Programación estructurada (C, Pascal, Fortran, ...)
- Programación orientada a objetos (Smalltalk, C++, Java, C#, ...)

❖ Lenguajes declarativos (funcionales y lógicos):

Los programas se implementan como conjuntos de funciones (lógicas) cuya evaluación nos dará el resultado deseado. Haskell, LISP, Erlang, Scala, Clojure, F#, Miranda, ...

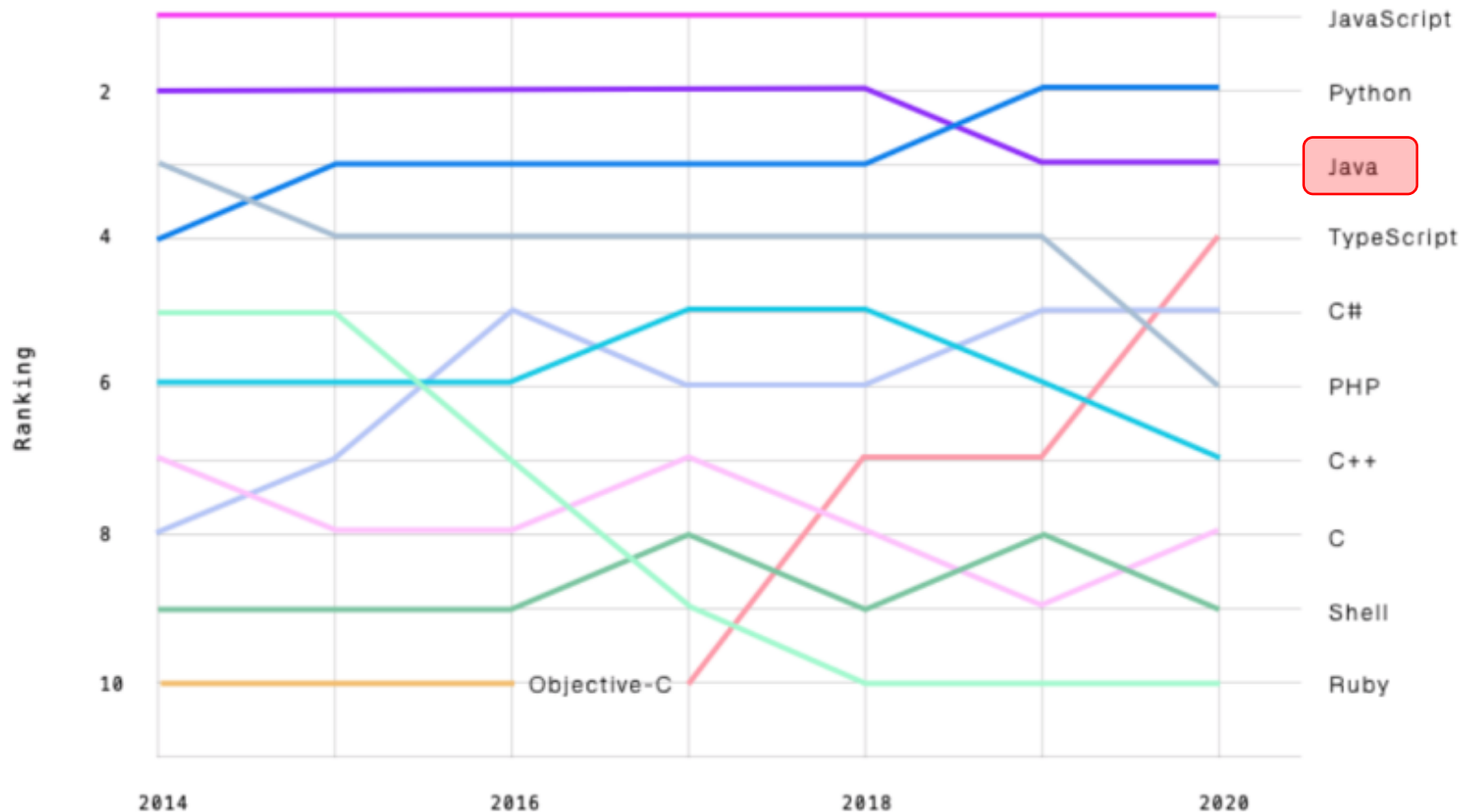
Lógicos: Prolog, Alf, Elf, Fish, Lambda Prolog, etc.

Evolución inicial de los lenguajes de programación imperativos



<https://www.youtube.com/watch?v=G-0kiQImaV8>

Los 10 lenguajes de programación más populares según **Github**

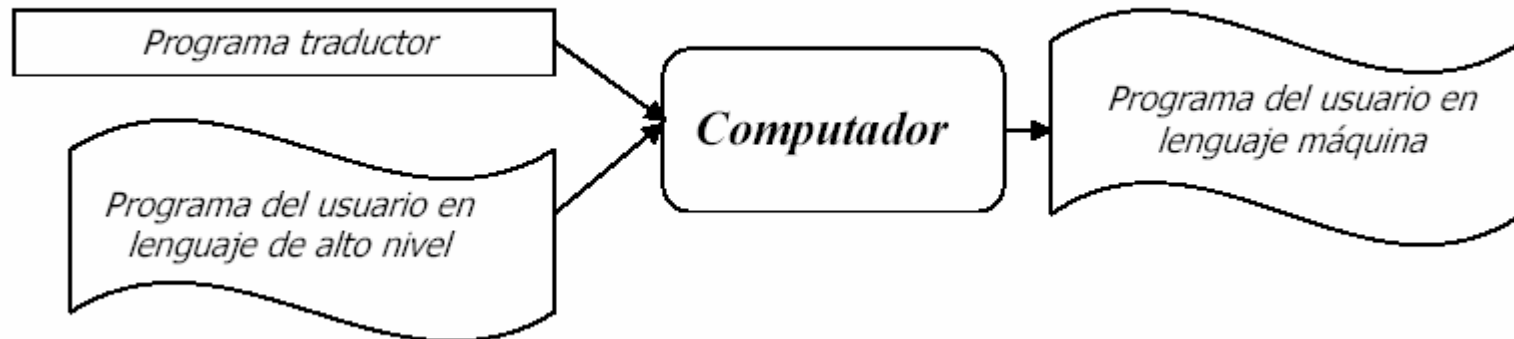


Otros lenguajes con impacto en la actualidad: Scala, Go, Swift, Groovy, Visual Basic, HTML, Perl, Kotlin, Objective-C, Rust, Ensamblador, R, ...

Tema 1. Fundamentos de Programación

➤ Traductores

Los traductores transforman programas escritos en un **lenguaje de alto nivel** en programas escritos en **código máquina**



Tema 1. Fundamentos de Programación

➤ Traductores

Tipos de traductores

- **Compiladores**



Generan un programa ejecutable a partir del código fuente (programa)

- **Intérpretes**

Van analizando, traduciendo y ejecutando las instrucciones del programa una a una. No se traduce una instrucción hasta que la ejecución de la anterior haya finalizado.

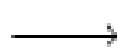
Tema 1. Fundamentos de Programación

➤ Código fuente + compilación = programa ejecutable

Código
fuente



Compilador



Programa
ejecutable

```
3 public class Circunferencia {  
4  
5     public static void main(String[] args) {  
6         double radio = 4.5;  
7         double longitud = 2 * Math.PI * radio;  
8         double area = Math.PI * Math.pow(radio, 2.0);  
9  
10        System.out.println("Radio = " + radio);  
11        System.out.println();  
12        System.out.printf("Longitud de la circunferencia = %.4f", longitud);  
13        System.out.println();  
14        System.out.printf("Area de la circunferencia = %.4f", area);  
15        System.out.println();  
16    }  
17 }  
18
```

Radio = 4.5

Longitud de la circunferencia = 28.2743
Area de la circunferencia = 63.6173

Tema 1. Fundamentos de Programación

➤ Código fuente + compilación = programa ejecutable



En un lenguaje de programación:
C, C++, Java, Python, etc

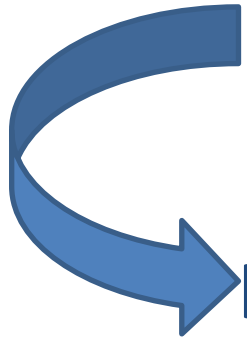
El programa se ejecutará en un sistema operativo concreto: Windows, macOS, Linux, etc

Tema 1. Fundamentos de Programación



Herramientas de programación

Editores de código fuente, depuradores, herramientas de compilación, ...



IDEs (entornos integrados de desarrollo)

Ejemplos: *Microsoft Visual Studio*, **Eclipse**, *Netbeans*, **Visual Studio Code**, IntelliJ Idea, BlueJ, jGRASP, Xcode, JDeveloper, Codenvy, etc.

Tema 1. Fundamentos de Programación

Most Popular Development Environments



All Respondents

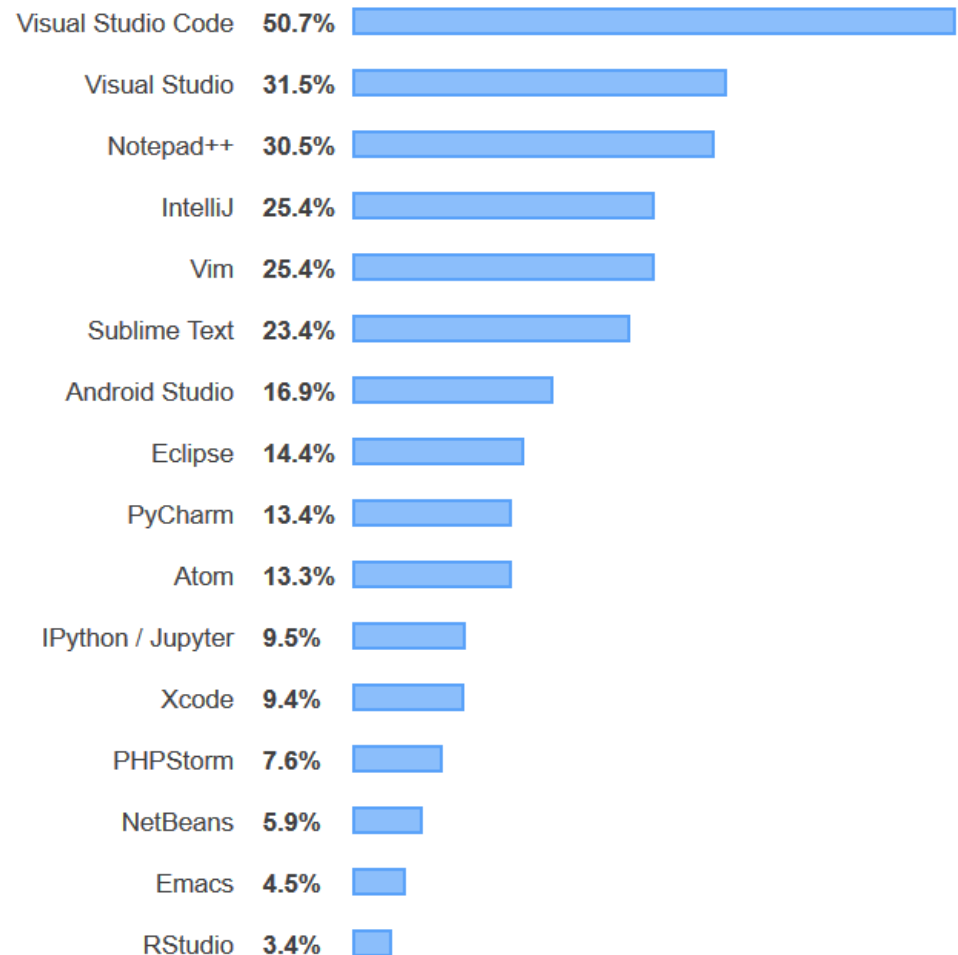
Web Developers

Mobile Developers

SRE/DevOps



Developer Survey Results
2019



<https://insights.stackoverflow.com/survey/2019>

Tema 1. Fundamentos de Programación

Por qué utilizar los Java IDEs

- Proporcionan plantillas de código predeterminada.
- Validan automáticamente la sintaxis Java.
- Son compatibles con la refactorización de código.
- Admiten la coloración de sintaxis.
- Permiten establecer puntos de interrupción.
- Ofrecen un depurador robusto.
- Proporcionan un editor de Java robusto y potente.
- Asistencia basada en Inteligencia Artificial

Tema 1. Fundamentos de Programación

Herramientas de programación (IDE-Eclipse)

Java - TemariIntroduccionProgramacion2016/src/org/ip/sesion01/Distancia.java - Eclipse - C:\WORKSPACES\Docencia_2016

File Edit Source Refactor Navigate Search Project Run Window Help

Package Explorer

- org.ip.sesion01 3
 - ConvertirCelsiusFahrenheit.java 3
 - Distancia.java 3**
 - DivisionPorCero.java 3
 - EnfriamientoViento.java 3
 - EnteroAleatorio.java 3
 - EntradaArgumentos.java 3
 - Hola.java 3
 - OperadoresRelacionales.java 3
 - OperadoresUnarios.java 3
- org.ip.sesion02 3
 - CodigoDefectuoso.java 3
 - DiaJuliano.java 3
 - DiaSemana.java 3
 - Ecuacion2Grado.java 3
 - EcuacionRecta.java 3
 - OperacionAritmetica.java 3
 - TarifaTaxi.java 3
- org.ip.sesion03 23
- org.ip.sesion04 22
- org.ip.sesion05 23
- org.ip.sesion06 24
- org.ip.sesion07 26
- org.ip.sesion08 30
- org.ip.sesion09 3
- org.ip.sesion10 3
- test 31
- JRE System Library [JavaSE-1.7]
- JUnit 4
- Documentos 27
- GrupoDocente 25

Distancia.java

```
16
17 public class Distancia {
18     public static void main(String[] args) {
19
20         int x = 2;
21         int y = 1;
22
23         // calculo de la distancia a (0, 0)
24         double dist = Math.sqrt(x * x + y * y);
25
26         System.out.println();
27
28         // salida de la distancia
29         System.out.println("La distancia del punto (" + x + ", " + y + ") al punto (0, 0) es "
30             + dist);
31     }
32 }
```

Task List

Find All Activate...

Connect Mylyn

Connect to your task and ALM tools or create a local task.

Outline

- org.ip.sesion01
 - Distancia 3**
 - main(String[]): void

Problems @ Javadoc Declaration Search Console History

<terminated> Distancia [Java Application] C:\ECLIPSE\GIPP2015\EclipseLuna\jdk1.8.0_31\bin\java.exe

La distancia del punto (2, 1) al punto (0, 0) es 2.23606797749979

org.ip.sesion01.Distancia.java - TemariIntroduccionProgramacion2016/src



Tema 1. Fundamentos de Programación



Ciclo de vida del software

Comprende las siguientes etapas:

- **Planificación**
- **Análisis (¿qué?):** Análisis de requisitos
- **Diseño (¿cómo?):** Estudio de alternativas
- **Implementación**
- **Depuración y pruebas**
- **Explotación: Uso y mantenimiento**

Tema 1. Fundamentos de Programación

- Algoritmos y programas
- **Introducción a la programación en Java**
- Datos y tipos de datos
- Tipos de datos primitivos en Java
- Variables
- Expresiones y sentencias
- Programas
- Estructuras de control
- Uso de subprogramas
- Métodos

Tema 1. Fundamentos de Programación



Introducción a la programación



- ☐ La plataforma de programación Java
- ☐ Aplicaciones y applets
- ☐ Características de Java

Tema 1. Fundamentos de Programación

➤ Qué es Java y objetivos principales

- **Java** es un lenguaje de programación basado en clases y **orientado a objetos**.
- **Objetivo:** permitir que los desarrolladores de aplicaciones escriban el código una sola vez y que ese código pueda ser ejecutado **en cualquier computadora**, WORA (Write Once, Run Anywhere)
- **Java** se puede combinar con otras muchas tecnologías como Spring, node js, Android, Hadoop, J2EE, etc. y nos permite crear aplicaciones completas, robustas, escalables, portátiles y distribuidas.
- **Java** también promueve la integración continua (Jenkins) y el control de versiones del código (Git, SVN), a través de pruebas unitarias, utilizando herramientas de testing (JUnit).

Tema 1. Fundamentos de Programación

➤ Por qué aprender programación con Java

- **Java** es uno de los principales lenguajes de programación en la actualidad y lo seguirá siendo al menos una década.
- Como **Java** es seguro y multiproceso, es perfecto para los servicios de gestión de transacciones y banca.
- El sistema operativo móvil como **Android** utiliza la API (Application Programming Interfaces) de Java.
- Los algoritmos del mercado de valores se basan en Java.
- Y más recientemente, la gestión de muchos de los grandes volúmenes de datos (**Big Data**) se tratan de forma sencilla mediante Java (Hadoop-MapReduce).

Tema 1. Fundamentos de Programación

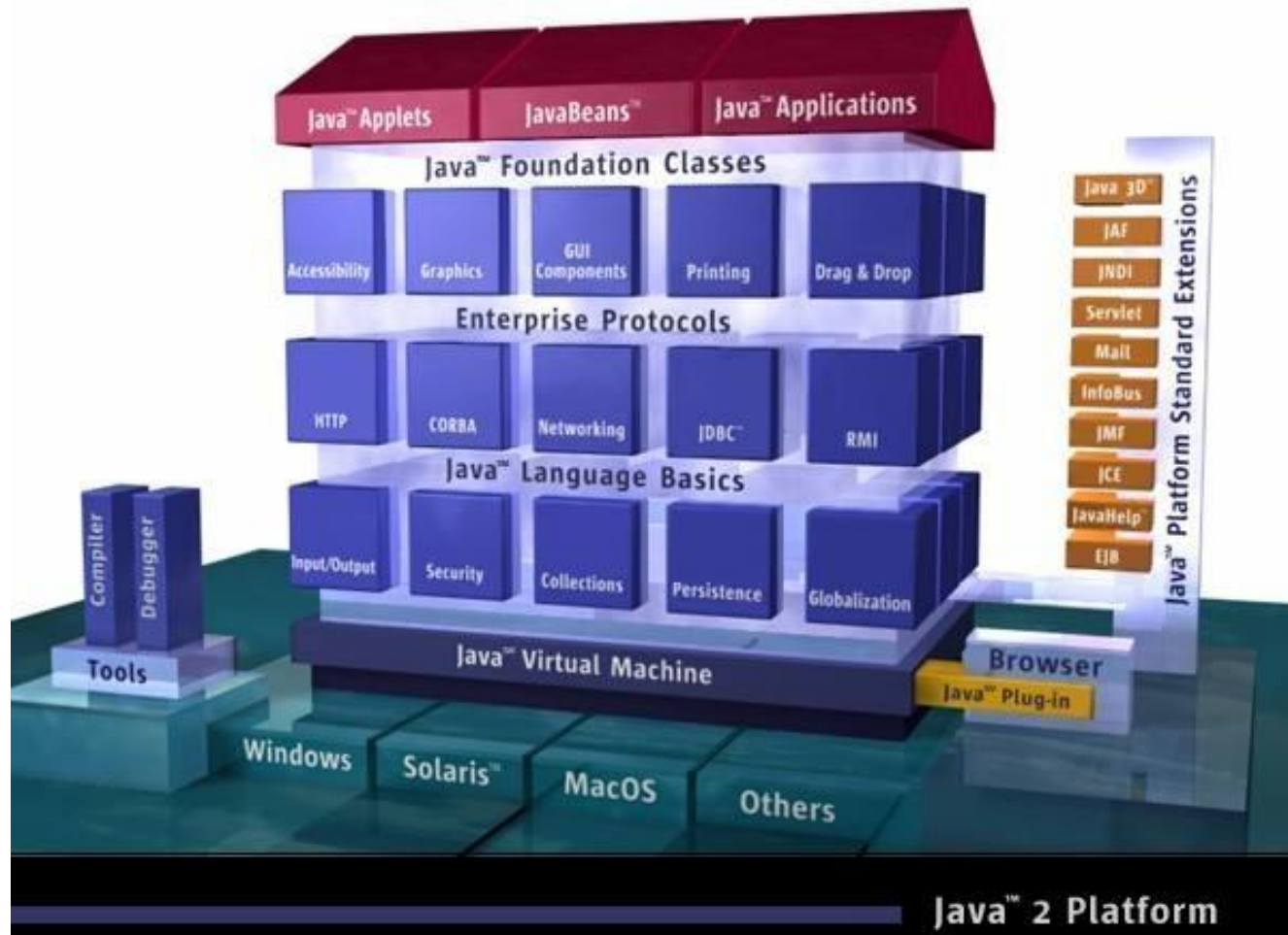


Papel de Java en el desarrollo de software

- Desarrollo web front-end: JavaScript
- Desarrollo web back-end: JavaScript, **Java**, Python, PHP, Ruby
- Desarrollo móvil: **Java**, Swift, Kotlin, C#
- Desarrollo de videojuegos: C++, C#, **Java**
- Aplicaciones de escritorio: **Java**, C#, C++, Python
- Aplicaciones financieras y banca: **Java**, C#, JavaScript, Cobol
- Desarrollo Big Data: Scala, **Java**, Python
- Blockchain y Criptomonedas: C, C++, Solidity, Python
- Programación de sistemas: C, Go, Rust

Tema 1. Fundamentos de Programación

➤ La plataforma de programación Java



Tema 1. Fundamentos de Programación

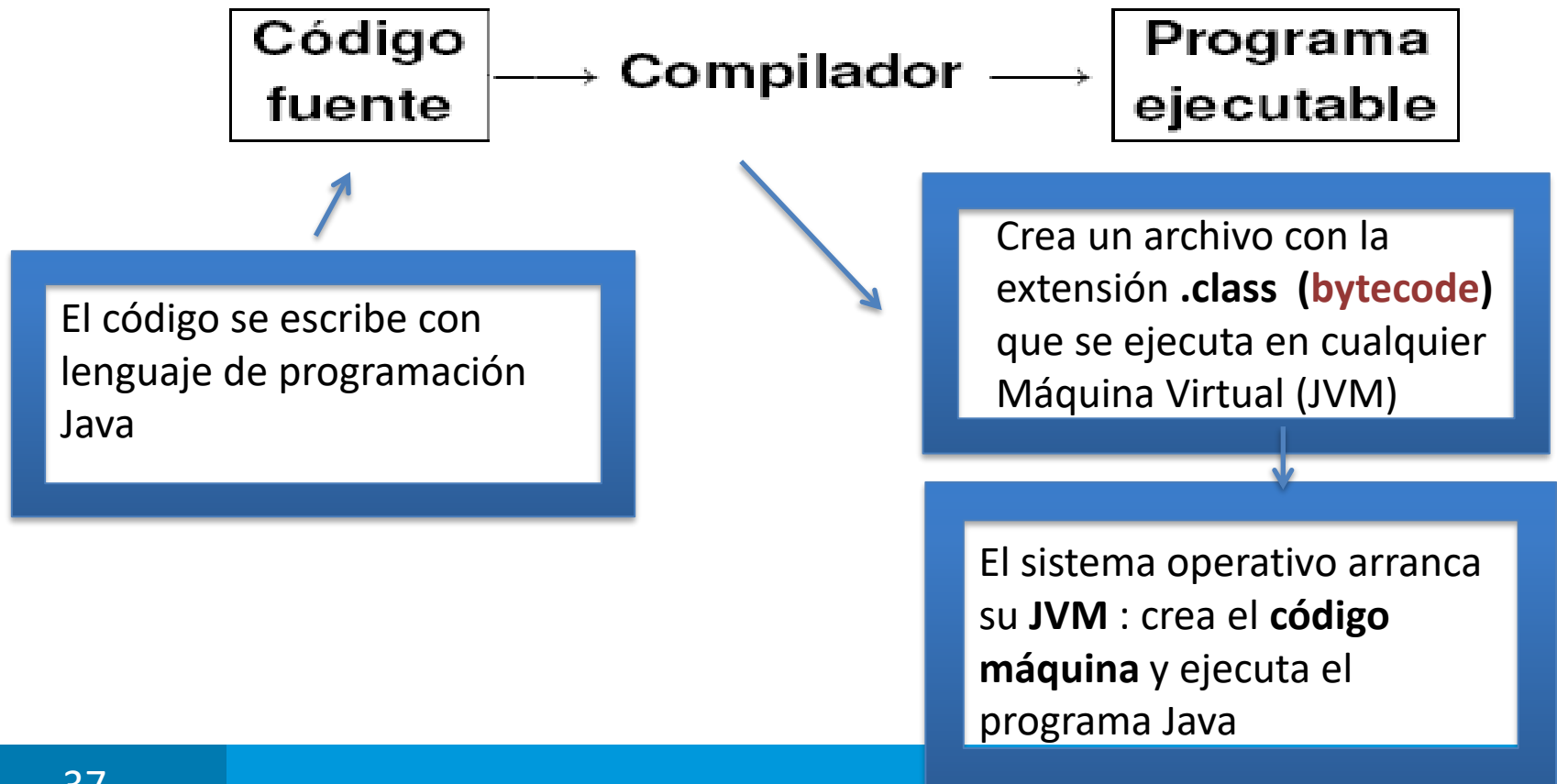
➤ La plataforma de programación Java



- La **máquina virtual Java** (JVM: Java Virtual Machine) Imprescindible para poder ejecutar aplicaciones Java.
- Las **bibliotecas estándar de Java** (Java Application Programming Interface = Java API) Amplia colección de componentes.
- El **lenguaje de programación Java**. Para escribir aplicaciones.

Tema 1. Fundamentos de Programación

➤ Código fuente java + compilación =
programa ejecutable en Máquina Virtual (JVM)



Tema 1. Fundamentos de Programación

➤ La plataforma de programación Java



La máquina virtual de Java

La máquina virtual Java



Tema 1. Fundamentos de Programación

➤ La plataforma de programación Java



La máquina virtual de Java

La máquina virtual Java



- El compilador de Java genera un código intermedio independiente de la plataforma (bytecodes).
- Los bytecodes pueden considerarse como el lenguaje máquina de una máquina virtual, la Máquina Virtual Java (JVM).
- Cuando queremos ejecutar una aplicación Java, al cargar el programa en memoria, podemos
 - a) Interpretar los bytecodes instrucción por instrucción
 - b) Compilar los bytecodes para obtener el código máquina necesario para ejecutar la aplicación en el ordenador (compilador JIT [Just In Time]).

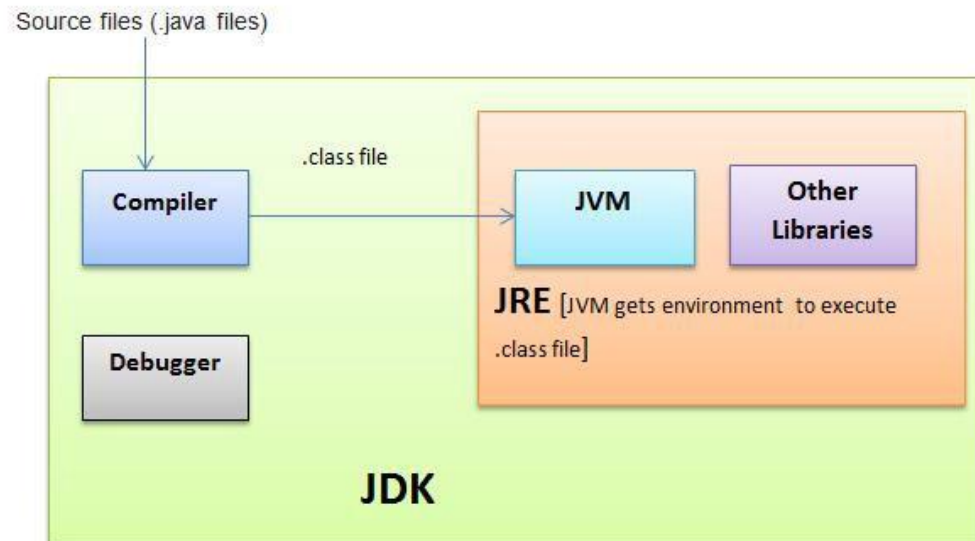
De esta forma, podemos ejecutar un programa escrito en Java sobre distintos sistemas operativos (Windows, Solares, Linux...) sin tener que recompilarlo, como sucedería con programas escritos en lenguajes como C.

Tema 1. Fundamentos de Programación

Componentes de Java



- **JDK:** Herramientas de desarrollo JRE, compilador (javac), archivos jar de bibliotecas (paquete y archivo), javadocs, etc.
- **JRE:** Implementación física de la JVM. Contiene los paquetes de la biblioteca y los archivos de soporte que JVM usa para ejecutar un programa.



Tema 1. Fundamentos de Programación

➤ La plataforma de programación Java

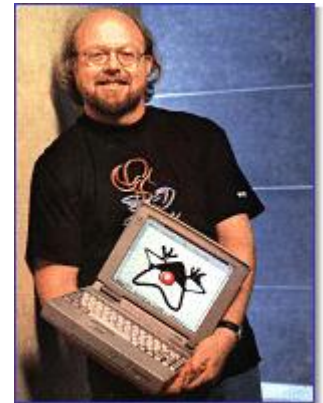


Historia de Java

La versión más difundida sobre el origen es la que presenta a **Java** como un lenguaje pensado para pequeños electrodomésticos. **Sun Microsystems** decidió intentar introducirse en el mercado de la electrónica de consumo y desarrollar programas para pequeños dispositivos electrónicos.



James Gosling



Tema 1. Fundamentos de Programación

➤ La plataforma de programación Java



Historia de Java

El creador de Java es **James Gosling**, cuyo objetivo era implementar una maquina virtual y un lenguaje de programación con una estructura y sintaxis similar a C++, pero más sencillo de utilizar.

El nombre se debe a un tipo de café cultivado en la **isla de Java**.

La primera versión se lanzó en **1995** (Sun Microsystems).

Más tarde, **Oracle** adquirió Sun Microsystems, y a partir de ahí Oracle mantiene la versión principal.

La última [versión de Java](#) es **Java 20** lanzada en Marzo de 2023.

Tema 1. Fundamentos de Programación

➤ La plataforma de programación Java



Uso típico de Java

- Aplicaciones (programas independientes)
- Applets (“pequeñas aplicaciones”) son programas diseñados para ejecutarse como parte de una página web.

Herramientas de programación en Java

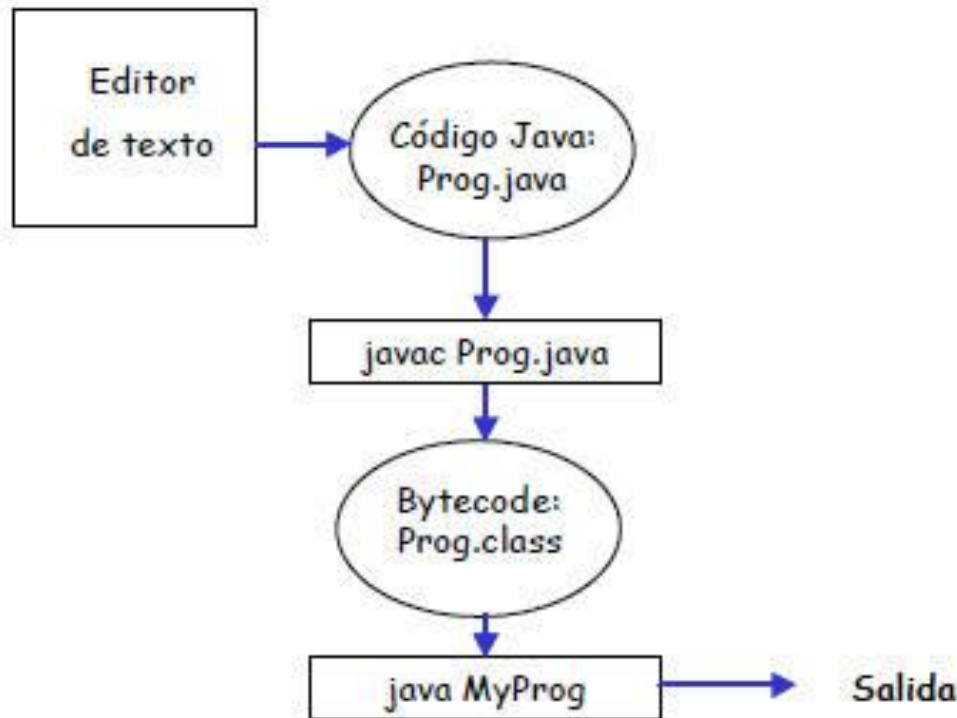
- **Java SDK [Software Development Kit]** <https://java.com>
 - ✓ Compilación de aplicaciones Java: **javac**
 - ✓ Ejecución de aplicaciones Java: **java**
 - ✓ Ejecución de applets: **appletviewer**
 - ✓ Generación de documentación: **javadoc**
 - ✓ Creación de archivos de distribución JAR [Java ARchives]: **jar**
 - ✓ Depuración de aplicaciones Java: **jdb**
 - ✓ etc.
- **Entornos integrados de desarrollo: IDEs** → **Eclipse** (<https://eclipse.org>) y **Apache NetBeans** (<https://netbeans.apache.org/>)

Tema 1. Fundamentos de Programación

➤ Aplicaciones Java



Creación y ejecución de aplicaciones Java



Tema 1. Fundamentos de Programación



Aplicaciones



Fases en la creación y ejecución de programas en Java

Fase I: Editor

- Se crea un programa con la ayuda de un editor. Se almacena en un fichero con extensión .java

Fase II: Compilador

- El compilador lee el código Java (fichero .java).
- Si se detectan errores sintácticos, el compilador nos informa de ello.
- Se generan los bytecodes, que se almacenan en ficheros .class

Fase III: Cargador de clases

- El cargador de clases lee los bytecodes (.class): Los bytecodes pasan de disco a memoria principal.

Fase IV: Verificador de bytecodes

- El verificador de bytecodes comprueba que los bytecodes son válidos y no violan las restricciones de seguridad de la máquina virtual Java.

Fase V: Intérprete de bytecodes o compilador JIT

- La máquina virtual Java (JVM) lee los bytecodes y los traduce al lenguaje que el ordenador entiende (código máquina).

Tema 1. Fundamentos de Programación

➤ Aplicaciones



Características clave de Java

- Java es **multiplataforma**
- Java es **seguro**
- Java tiene un amplio conjunto de **bibliotecas** estándar que facilita la codificación o programación
- Java incluye una biblioteca portable para la creación de **Interfaces Gráficas** de Usuario (AWT y JFC/Swing)
- Java simplifica algunos aspectos al programar
- Permite **recolección automática** de basura (memoria)
- Admite **multiprocesamiento y concurrencia**

Tema 1. Fundamentos de Programación



Datos y tipos de datos

- 
- ☐ Dato
 - ☐ Tipo de dato
 - ☐ Codificación de los datos en el ordenador

Tema 1. Fundamentos de Programación

➤ Dato

Representación formal de hechos, conceptos o instrucciones adecuada para su comunicación, interpretación y procesamiento por seres humanos o medios automáticos.

➤ Tipo de dato

Especificación de un dominio (rango de valores) y de un conjunto válido de operaciones a los que normalmente los traductores asocian un esquema de representación interna propio.

Tema 1. Fundamentos de Programación

Clasificación de los tipos de datos

- En función de quién los define:
 - ✓ Tipos de datos estándar
 - ✓ Tipos de datos definidos por el usuario

- En función de su representación interna:
 - ✓ Tipos de datos escalares o simples
 - ✓ Tipos de datos estructurados

Tema 1. Fundamentos de Programación

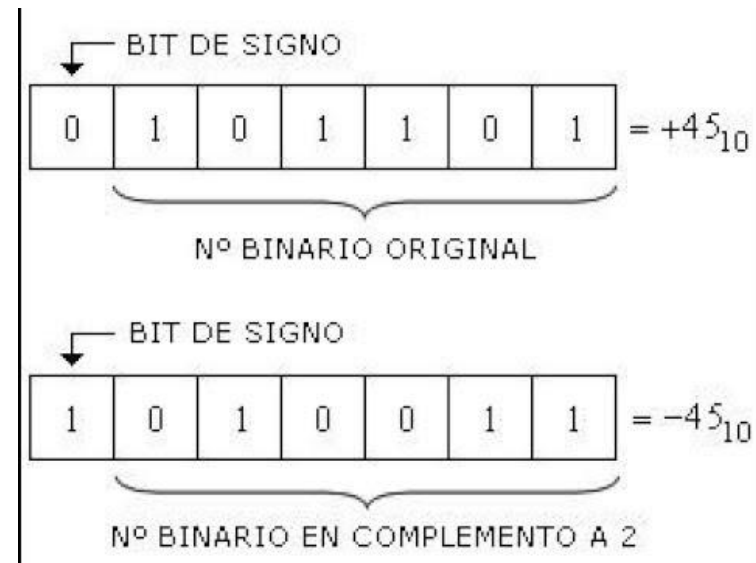
➤ Codificación de los datos en el ordenador

En el interior del ordenador, los datos se representan en **binario**.

Un **bit** nos permite representar 2 símbolos diferentes: **0** y **1**

En general,

N	2^N
1	2
2	4
3	8
4	16
5	32
6	64
7	128
8	256
9	512
10	1024
11	2048
12	4096
13	8192
14	16384
15	32768
16	65536



Tema 1. Fundamentos de Programación

➤ Codificación de los datos en el ordenador

NÚMEROS ENTEROS

Ejemplo: Si utilizamos **32 bits** para representar números enteros, disponemos de **2^{32}** combinaciones diferentes de 0s y 1s:

4 294 967 296 valores.

Como tenemos que representar números negativos y el cero, el ordenador será capaz de representar:

del **-2 147 483 648** al **+2 147 483 647**.

Con 32 bits no podremos representar números más grandes.

!!! 2 147 483 647 + 1 = -2 147 483 648 !!!

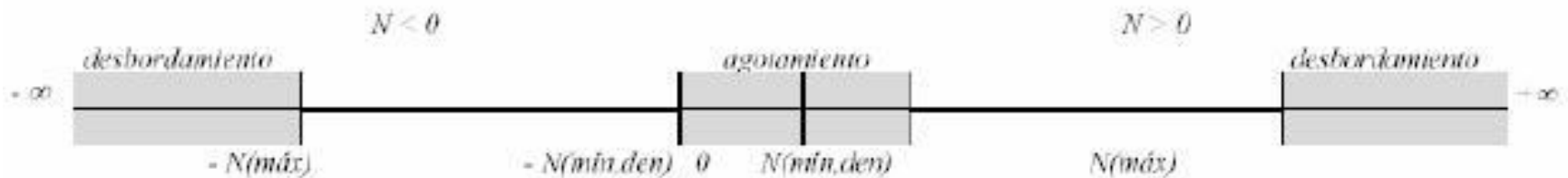
Tema 1. Fundamentos de Programación

➤ Codificación de los datos en el ordenador

NÚMEROS REALES (en notación científica)

(+ | −) mantisa $\times 2^{\text{exponente}}$

- El ordenador sólo puede representar un subconjunto de los números reales (números en coma flotante)
- Las operaciones aritméticas con números en coma flotante están sujetas a errores de redondeo.



Tema 1. Fundamentos de Programación

Tipos primitivos en Java

Tipo	Representación / Valor	Tamaño (en bits)	Valor mínimo	Valor máximo	Valor por defecto
boolean	true o false	1	N.A.	N.A.	false
char	Carácter Unicode	16	\u0000	\uFFFF	\u0000
byte	Entero con signo	8	-128	128	0
short	Entero con signo	16	-32768	32767	0
int	Entero con signo	32	-2147483648	2147483647	0
long	Entero con signo	64	-9223372036854775808	9223372036854775807	0
float	Coma flotante de precisión simple Norma IEEE 754	32	$\pm 3.40282347E+38$	$\pm 1.40239846E-45$	0.0
double	Coma flotante de precisión doble Norma IEEE 754	64	$\pm 1.79769313486231570E+308$	$\pm 4.94065645841246544E-324$	0.0

Tema 1. Fundamentos de Programación

➤ Codificación de los datos en el ordenador

Representación de textos

- Se escoge un **conjunto de caracteres**: alfabéticos, numéricos, especiales (separadores y signos de puntuación), gráficos y de control (por ejemplo, retorno de carro).
- Se codifica ese conjunto de caracteres utilizando **n bits**.
- Por tanto, se pueden representar hasta **2^n símbolos** distintos.

Tema 1. Fundamentos de Programación

Codificación de los datos en el ordenador

Ejemplos de códigos normalizados

ASCII (American Standard Code for Information Interchange)

- ANSI X3.4-1968, 7 bits (128 símbolos)
- ISO 8859-1 = Latin-1, **8 bits** (256 símbolos)

		0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
00	0	NUL	SOH	STX	ETX	EOT	ENQ	ACK	BEL	BS	HT	LF	VT	FF	CR	SO	SI
10	16	DLE	DC1	DC2	DC3	DC4	NAK	SYN	ETB	CAN	EM	SUB	ESC	FS	GS	RS	US
20	32	SP	!	"	#	\$	%	&	'	(	)	*	+	,	-	.	/
30	48	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
40	64	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
50	80	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
60	96	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
70	112	p	q	r	s	t	u	v	w	x	y	z	{		}	~	DEL
80	128																
90	144																
A0	160		¡	¢	£	¤	¥	¦	§	¨	©	ª	«	¬	®	¯	
B0	176	°	±	²	³	´	µ	¶	·	¸	¹	º	»	¼	½	¾	¿
C0	192	À	Á	Â	Ã	Ä	Å	Æ	Ç	È	É	Ê	Ë	Ì	Í	Î	Ï
D0	208	Ð	Ñ	Ò	Ó	Ô	Õ	Ö	×	Ø	Ù	Ú	Û	Ü	Ý	Þ	ß
E0	224	à	á	â	ã	ä	å	æ	ç	è	é	ê	ë	ì	í	î	ï
F0	240	ð	ñ	ò	ó	ô	õ	ö	÷	ø	ù	ú	û	ü	ý	þ	ÿ

Tema 1. Fundamentos de Programación

➤ Codificación de los datos en el ordenador

Ejemplos de códigos normalizados

UNICODE, ISO/IEC 10646

Es el estándar de codificación de caracteres universal utilizado para la representación de texto

Zona	Códigos		Símbolos codificados	Nº de caracteres
A	0000	0000 00FF	Latin-1	256
			otros alfabetos	7.936
		2000	Símbolos generales y caracteres fonéticos chinos, japoneses y coreanos	8.192
I	4000		Ideogramas	24.576
O	A000		Pendiente de asignación	16.384
R	E000 FFFF		Caracteres locales y propios de los usuarios. Compatibilidad con otros códigos	8.192

Ejemplos de caracteres
Unicode



Carácter alfabético latino
"A" (U+0041).



Sílaba devanagari "Aum"
(ॐ) (U+0950).



Ideograma chino "yue"
(月)(U+6708).



Emoji smiling face
with heart-eyes

[U+1F60D](#)

Tema 1. Fundamentos de Programación

➤ Codificación de los datos en el ordenador

Ejemplos de códigos normalizados

UTF-8 (8-bit Unicode Transformation Format)

- Formato de **codificación** de caracteres Unicode e ISO/IEC 10646
- Utiliza símbolos de longitud variable: de 1 byte a 6 bytes
- Incluye la especificación US-ASCII de 7 bits, la codificación de cualquier texto ASCII es directa
- Definido como estándar por la Internet Engineering Task Force (IETF) para uso en Web
- Ahorra espacio de almacenamiento para textos en caracteres latinos, donde los caracteres incluidos en US-ASCII son comunes
- Permite la **compatibilidad** del código fuente **entre sistemas operativos**:
Windows, Linux, MacOS.

¡MUCHAS GRACIAS!



UNIVERSIDAD DE ALMERÍA

